

# Best Time To Vacation

Wander Bravo, Piter Garcia, Anshika Garg

## ABSTRACT

The aim of this project is to learn the Big Data and Data Mining techniques to implement them in a project chosen by our team. The name of our project is "*Best Time To Vacation*", and we will be using the weather datasets of the United States to analyze, compare, and show the weather trends in particular areas. Thus, we will be able to predict the weather and precipitation of those places requested by the user on a designated time range. Moreover, we will be able to suggest places that most users like. As a result, we have designed an interface that will allow users to monitor the weather of a city, including its precipitation, for short-term vacationing purpose.

## 1. OVERVIEW

The logical structure of our project is composed of three main sub-structures, which better describe the milestones achieved throughout our project development. These sub-structures are classified as Design, Architecture and Implementation. Each section explains in detail the main steps taken to model and structure a database with our interface to meet the project specifications. In order to better organize our documentation we divided this into two main sections that contain a summary of the work process done in our project. These two sections are Database and Website, and their development can be described as follows:

In the database section three main sub-sections take place in order to make the database usable. These sections include Cleaning, Design, and Tables. First, in the formatting section, we will elaborate how our datasets are formatted, the process taken to make our datasets easy to read by avoiding noisy data and giving them an appropriate format. Second, in the Design section, we will explain the design of our database in terms of appearance, structure and performance. Lastly, in the Tables section, we will describe the process taken to load and distribute our datasets into their appropriate tables. Also, we will explain how each table is generated as well as their functionality.

The Website development process is explained within the design, registration, and features sub-sections, respectively. First, in the design sub-section, our interface is designed according to the appearance, structure, and performance we wanted our project to achieve. Second, in the registration sub-section, we will describe the process taken to create the appearance, structure, as well as techniques used to allow users to access our system in an efficient manner. Third, in

the features sub-sections, we describe the functionality for each of the features to be performed through the interface via our database and system analysis.

In particular, we will explain some tools and techniques used to achieve our goals in each one of the sub-sections explained in the Database and Website sections. These sub-sections are explained separately in each sub-structure as the process taken in each one varied.

## 2. DESIGN

In this project, the operational process of our design meets the objective of a high quality development which responds to real life scenarios in big data. The focus of this step is the development of an acceptable functional solution to the requirements of our project with an exterior design solution that is compatible with our database, interface, and our goal as a whole. The progress sets of our design were formally modified, implemented, and reviewed by our group as they were issued at the completion of our schematic design, design development, and during the development of our database. A successful design development was needed in order to coordinate the activities of users and project architecture to meet our goal, as these are important for our overall project development. In fact, our project design process typically involves the sequence of activities described in the following sub-sections.

### 2.1 Database

The datasets consist of the temperature max/min, precipitation max/min, wind speed, longitude and latitude (position about 1/8 degrees apart) of some locations. These datasets have been collecting weather information for the past forty-four years, which has helped us analyze the trends of some areas over the past years and made our results more reliable. The main goal was to design a database, from which we can easily generate tables for the cities, cities' s weather information, and user' s cities selections as shown in the image below.

| Tables        | Attributes |           |         |           |           |                  |                  |
|---------------|------------|-----------|---------|-----------|-----------|------------------|------------------|
| Person        | name       | password  |         |           |           | minTemperature   | maxTemperature   |
| Search        | area       | startDate | endDate |           |           |                  |                  |
| Precipitation | year       | month     | date    | longitude | latitude  | minPrecipitation | maxPrecipitation |
| Temperature   | year       | month     | date    | longitude | latitude  | min temperature  | maxTemperature   |
| Place         | area       |           |         | longitude | latitude) |                  |                  |

Figure 1: Design model 1.

## 1. Formatting

In order to make a better use of our database we have formatted each dataset to remove the redundant and or noisy data as well as to help our system to run more efficiently. The main goal was to make querying for the weather information easy and reliable.

- **Appearance**

The main purpose for the formatting process was to make our database look as the images below. In this way, we are able to read the formatted data files according to the requirements of our project. This process allows us to easily relate each one of the input dates to the corresponding city and its temperature, within the given range, to approximate the city's weather condition within the given dates.

- **Structure**

In order to create the desired appearance described above, we needed to make our database to go through some steps to make this structure possible. First, the datasets were compressed and therefore we needed extract them by using the little Endian format, which was a very difficult process to include in our code. Second, we sorted the cities we needed to separate them from those we did not plan to include in our database. Lastly, each dataset was formatted to relate the weather, user, and longitude and latitude tables to its corresponding city by means of an id, and to be able to match such city weather record's dates to the user given dates.

- **Performance**

The datasets we obtained were huge and very difficult to handle. The formatting process was a very important step in our project to improve and speed up every process. At first, it was very difficult for us to test our features due to the fact that the system was very slow and overloaded with the huge amount of data that it was trying to process. This is why we proceeded to exclude a huge amount of cities and only work with some cities to make testing more efficiency and reliable.

## 2. Design

We designed the appearance and structure of our database to perform according to the datasets obtained as mentioned above.

- **Appearance**

The purpose of our design was to allow the tables in our database to relate the input dates to the corresponding city and its temperature, within the given range, to approximate the city's weather condition within the given dates, as shown in the images.

- **Structure**

The main structure of our database was based mainly on the Temperature, City, Longitude-and-Latitude Tables. The structure of these were developed by tools that are explained in the architecture section of this project. An example of the

design of this structure can be seen in the image below.

User's table had a different structure to this since this table is populated as users registered. The structure of this was developed by tools that are explained in the architecture section of this project. The structure design of this table can be seen in the image below.

- **Performance**

The performance of each table in our database is related to the user's table. After the user has signed into our system, this will populate the user's table accordingly. The content of this will access to the other tables, such as the city, longitude and latitude, and weather tables. Once it has gathered the necessary data it will proceed to do data mining to approximate the weather condition of a city, selected by the user, within the dates. The way that this performance takes place is explained in detail in the architecture section of this project.

```
-- Table structure for table `city`
--

CREATE TABLE `city` (
  `id` int(11) NOT NULL,
  `name` varchar(25) NOT NULL,
  `latitude` float NOT NULL,
  `longitude` float NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

Figure 2: Design model 1.

```
--
-- Table structure for table `data`
--

CREATE TABLE `data` (
  `id` int(11) NOT NULL,
  `year` int(11) NOT NULL,
  `month` int(11) NOT NULL,
  `day` int(11) NOT NULL,
  `prec` int(11) NOT NULL,
  `tmax` int(11) NOT NULL,
  `tmin` int(11) NOT NULL,
  `wind` int(11) NOT NULL,
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

Figure 3: Design model 1.

## 3. Tables

The main tables used to create our database are the Data and City tables. These tables were created, after uncompressing the data, with an algorithm we coded in java. This process is better explained in the architecture section, however here will explain how this process takes place.

- **Appearance**

In order to load our tables into our system it was necessary for us to have the two tables mentioned above formatted.

- **Structure**

```
--
-- Dumping data for table `city`
--
INSERT INTO `city` (`id`, `name`, `latitude`, `longitude`) VALUES
(1, 'Montgomery', 32.23, -86.22),
(2, 'Bridgeport', 41.11, -73.11),
(3, 'Washington', 38.51, -77.2),
(4, 'Miami', 25.48, -80.16),
(5, 'Orlando', 28.33, -81.23),
(6, 'Dublin', 32.2, -82.54),
(7, 'Atlanta', 33.39, -84),
(8, 'Baltimore', 39.11, -76.4),
(9, 'Boston', 42.22, -71.2),
(10, 'Trenton', 40.13, -74.46),
(11, 'Albany', 42.45, -73.48),
(12, 'Raleigh', 35.52, -78.47),
(13, 'NYC', 40.47, -73.58);
```

Figure 4: Design model 1.

The structure of each table should be as following. First, the City table, is composed by latitude, longitude, and cityName. Lastly, the Data table, which is composed by the date, precipitation, temperature, and wind of each city. This is the structure we use to design our database as shown in the image above.

- **Performance**

The City and Data table is used by the user's table to implement the features of our system. This include, weather approximation and to suggest cities that the user may like according to the user's behavior.

## 2.2 Interface

We have designed an interface that works as a platform to shape and implement our project according to our expectations. In order to fulfill the design and project patterns, we are going to break down our approach into the following subsections.

### 1. Design

The way we have designed our interface is according to the database design. We have created a set of features that users will be able to use in our interface through our database implantation. In order for users to be able to access these features the first must be able to sign in into our system. This is why we have described our interface section in the following sub-sections.

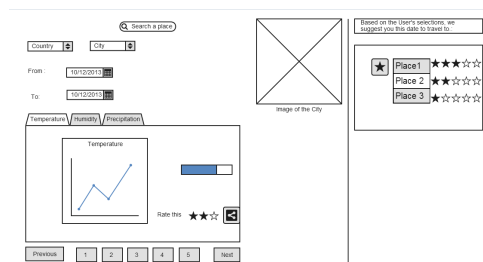


Figure 5: Design model 1.

- **Appearance**

An example of the appearance of our interface, or interface, design is in the above image. The appearance of our interface is very well influenced

by the architecture of your project. Therefore, even though the design of our interface appearance may not concede with the implementation the features are the same.

- **Structure**

The tools that we have used give our interface the structure needed to meet our project specifications. For this project, PostgreSQL is the database engine underlined by our management system to create, read, update and delete data from the database. Moreover, we have included other database and web development software that have helped us to model and process our database faster and easier. The application-programming interface that we are using, to allow users to interact with our underlying engine without going through the user interface of our system, is Django. We are also making use of tools such as Python and HTML. These tools are described in detail in the architecture section of our project, and examples of these are given to show the reader how the structure of our interface took place.

- **Performance**

The platform that we are using to manage our database is Django. The installation and upgrading process is very easy since we have the ability to automatically install database adapters and other dependencies. Using Django in our management systems has allowed us to extract valued content from our data sets and build a high performing web applications system in an elegant and efficient way. The following factors have been improved with the use of Django. First, we can define our data models much easier with Python that provides us with a dynamic database access APIA, which we have been writing with SQL. Second, it provides us with an automatic admin interface that adds and updates users and user's content automatically. Third, it supports Multi-language applications. This is essential for our implementation since is mostly for vacation purpose. Finally, we can always use cache frameworks to increase future performance.

### 2. Features

All features of our project were designed towards our main goal. This is to help user to decide which places to visit to vacation. In order to this, we need to provide users with features that allow them to search for a place, monitor the approximate weather information, and have a list of suggested cities users make want to visit according to their behavior. The design of these features can be described as following:

- **Registration**

In the signing process the user needs to log in by providing a username and a password. This will assign each user with a unique id and password. An example of this feature can be seen in the image above. Moreover, we have an application for users to log into our system through Facebook. This feature facilitates users to utilize our system in an easier manner as well as to improve our system as follows. First, helping us to increase our

# Best time to vacation

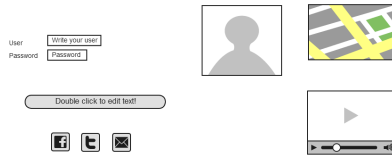


Figure 6: Design model 1.

project registration. Second, providing us with app id and app secret id, this way the user does not have to worry about creating a new username and or password. Overall, this feature could potentially help us to integrate more features and for future improvements.

- **Search**

This is very important feature since it is through which users are able to search and find cities of their interested. This feature will automatically add the searches done by the user, the list of searches or table is later used to suggest the user a list of cities that he or she may like. More detail of the structure of this feature is covered in the architecture section of our project. The image above is an example of this feature.

- **Graph**

In the graph feature there is function that we use to approximate the weather information of a city. Once the weather data was approximated this data is abscess and display by the graph feature. The process in which we make use of the weather data to make the approximation is better explain in the architecture section of our project.

- **Suggestion List**

This feature shows a graphical representation of the approximated weather and precipitation of the designated city that the user has searched for. In this feature, we use the table that stores all the searches done by the user. We use the data in this table such as user's information and the weather information of the cities. After all this information is gather we create another table that will contain the suggested cities that the user may want to visit. The structure of this features is quite explained in the architecture section of our project.

## 3. ARCHITECTURE

It was critical to model the right architecture in order to succeed in our system design. The architecture system representation was essential for the description and analysis of the high level properties of our system. This essential tenet forced us to consider alternatives due to the architecture of our project. The architecture of our project specifies the overall structure of our system. This includes the communication, synchronization, and data access, which are

described below in the database and interface sub-sections. The database contains the assignment of functionality for our designed features, data corporal distribution, composition of designed features, scaling and performance, and data design modifications. In contrast, the interface sub-section includes the body of work for our module interconnection features, templates and frameworks for our system which serve the needs of our database domains, and formal models of component integration mechanisms. Also, this includes the body of work for this project such as the module interconnection features, templates and frameworks for our system which serve the needs of our database domains, and formal models of component integration mechanisms. The two following sub-sections will illustrate the architecture process done throughout our project development.

### 3.1 Database

#### 1. Formatting

In the formatting process of our project we used some algorithm techniques to model our data to meet the project requirements. These algorithms along with some software tools that we used are being explain below.

- **Appearance**

The appearance of the database was obtained through algorithms such as the little Edian, and our own code implementation to reduce the datasets to only 20 years of weather information and to reduce the number of cities.

```
unsigned short int *prec;
short int *array1, *array2, *array3;

for (x=0;x<lines/4;x++) {
if(fread(&prec[x],sizeof(unsigned short int),1,fp) != 1)
    printf("read prec[x] err rec %d %s\n", x,file);
if(fread(&array1[x],sizeof(short int),1,fp) != 1)
    printf("read array1[x] err rec %d %s\n", x,file);
if(fread(&array2[x],sizeof(short int),1,fp) != 1)
    printf("read array2[x] err rec %d %s\n", x,file);
if(fread(&array3[x],sizeof(short int),1,fp) != 1)
    printf("read array3[x] err rec %d %s\n", x,file);

prcp = (float) (prec[x])/40.0;
tmax = (float) (array1[x])/100.0;
tmin = (float) (array2[x])/100.0;
wind = (float) (array3[x])/100.0;
}
```

Figure 7: Design model 1.

Little Edian is a compression format algorithm that we used to uncompressed the datasets mentioned in the design section of our project. We had access to the algorithm through Washington University Website as in the reference, and a demonstration is shown in the image above.

We implemented in java the code shown in the image above. This code takes the data and reduce it to a 20 years of weather data.

- **Structure**

In order to use our data correctly we needed to model this in a way we could easily loaded it to our database and use it for querying. To do this, we have used our own code implementation that creates the city and data table explained in the

```

while ((line=br.readLine())!=null)
{
    String [] parts = line.split("\t");
    int len = parts.length;
    fileOut.print (id+"\t"+month+"/"+date+"/"+year+"/"+t");
    if(date == day[month-1])
    {
        month++;
        date = 1;
        if(month > 12)
        {
            year++;
            month = 1;
        }
    }
    if(month == 2 && year%4 == 0)
    {
        day[month-1] = 29;
    }
    else
    {
        day[1] = 28;
    }
    for(int i = 0; i < len-1; i++)
    {
        fileOut.print (parts[i]+"\\t");
    }
    fileOut.print (parts[len-1]+"\\n\\n");
}

```

Figure 8: Design model 1.

designed section. These tables are created by the algorithm shown in the image above. This algorithm has to parts, the first one is to create the city table and the second one is to create the data table. An image of these two is shown in the images above.

#### • Performance

The reduction of the datasets and the formatting of these made easier the querying process. The algorithm mentioned in the structure of this section was necessary in order to do our testing procedures because the datasets were huge, and also we did not need more than 20 years of weather data to approximate the weather of a city.

## 2. Design

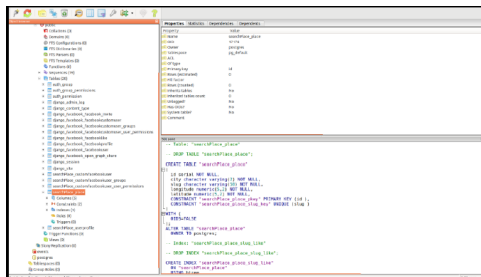


Figure 9: Design model 1.

#### • Appearance and Structure

The structure of our database was modeled through the pgAdminIII work platform. This work platform helped us to generate the tables that we needed to make the functionalities of each features, mentioned in the design section of this report, possible. Through this application we were able to generate the city and data tables, which

are very essential for our implementation. A demonstration of the work platform is shown in the image above. The structure of the

#### • Performance

The pgAdminIII platform devises a query plan for each query that is given to generate the tables. The generation of these tables will decrease the querying time, and gives us a close approximation in quick step.

## 3. Tables

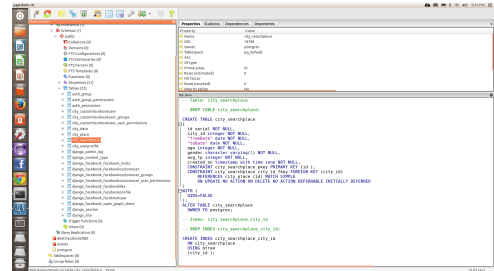


Figure 10: Design model 1.

#### • DATA

The data table consist of weather information of the cities, these are temperature max/min, precipitation and wind speed. These weather information corresponds to each city id, and the data is from 1980 to 2013. The schema of this table is shown in the image above.

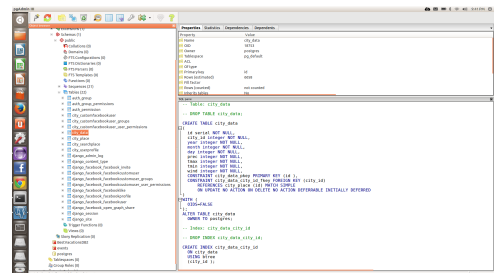


Figure 11: Design model 1.

#### • CITY

The data table for the city contains only the id, the longitude, and latitude of each city. The id is used reference to the Data table. The schema of this is shown in the image below.

#### • SEARCH

In the search place table, we have assigned to each area, which we named city in our query, a unique id through which we can distinguish it from the other areas. Each city has its own longitude and latitude and these values are provided by our database. The slug is required by our system, this is a way of generating a valid URL using



the data already obtained, an example of this table is shown above.

In the image below, we show how our search place table looks like. We have assigned to each area, which we named city in our query, a unique id through which we can distinguish it from the other areas. Each city has its own longitude and latitude and these values are provided by our database. The slug is required by our system, this is a way of generating a valid URL using the data already obtained.

- **USER**

The user table is generated through Django by importing the data from Facebook to the user table. This table can later be used in the pgAdminIII work platform to query accordingly.

- **city data**

contains the data of temp, wind, prec that matches with the cities

- **city place:**

contains the name, the long and lat for each city.s

- **city searchplace**

this is the table that we are filling to do the data mining, since we are saving:

city, fromDate, toDate (range of date), age, gender, avg tmp(the average of the temperature for the range), created on (the dateTime when the user did the search

- **city userprofile:**

contains the data of temp, wind, prec that matches with the cities

Based on this parameters, next time the user does a search, we are going to suggest cities, based on the age, the gender, and the average of temperature.

this table contains the information from the user, extracted from Facebook.

## 4. Features

- **SIGN IN**

The table Person is very necessary for the registration process, this can be explained as follows. First, to register the user we need the user's name and password and these data is only used for registration and user identification process. Second, we need the temperature that the user desires in a place of vacation. All these data is gathered and saved in a table named Person, and the reason why is important is because this table allows us to identify users and predict users behavior according to their search history. The other tables are better explained in section 2.2. The registration process works in the following manner. The user needs to log in by providing a username and a password. We create a temporary table of this

user to check, then we check if the user is in our system. If the user is already in our database we just update the user's table. Otherwise, we add the temporary table as a new user and update our homepage.

## 3.2 Interface

### 1. Design

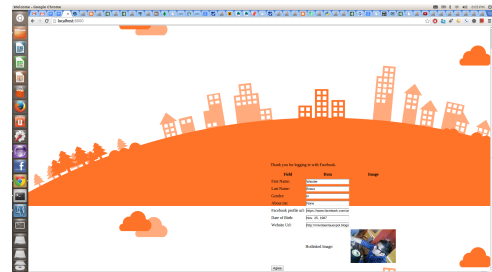


Figure 12: Design model 1.

- **Appearance**

An example of our interface look is shown in the image below. The features that we display are sign in, search, graph, and suggested cities. The design structure of each of these is covered below.

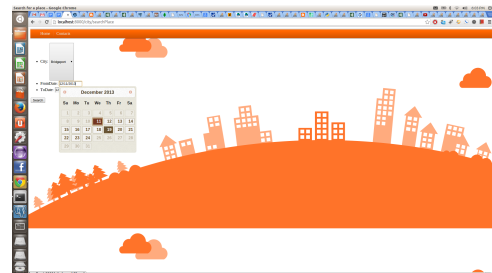


Figure 13: Design model 1.

- **Structure**

The structure of our interfaced is modeled by Django through its queries techniques. We use these techniques to populate the user table, to query the other tables in our database, and to perform some data mining techniques. Django provides for an intuitive way of representing the database in python. It uses a model class for a database table where, an instance of a class represents a record of the table, as shown in the image below.

image

The class below represents a table of the database named Blog in python.

Creating Objects

To create an object we would need to instantiate using keyword arguments to the model class and

then, save the changes. The class needs to be imported by using the python path to the class. Assuming models live in a file mysite/blog/models.py, This code below adds a record to Blog.

#### Updating Objects

This is an intuitive task where, we perform the updating in the class. Next, ask the database to save the changes. The example below changes the name of the instance of the class b5.

#### Retrieving Objects

This is done using the Query set on the manager. Queryset in Django is equivalent to a select statement. In the example below object represents the default manager and Blog() represents the query set with a few filters to filter out the results.

Next, a number of filters can be used on the resultant query. Keywords like filter and exclude are used for that. The keyword all is used if all the results are required. In case we require only a single output the keyword get is used. We could also limit the results such that we get a range of results from the table.

The architecture of our system can be better improve by understanding the complexity of our system. We recognized common paradigms which allow high level relationships among systems to be understood. In this way, we will be able to build a new system as variations on our current systems.

## 4. IMPLEMENTATION

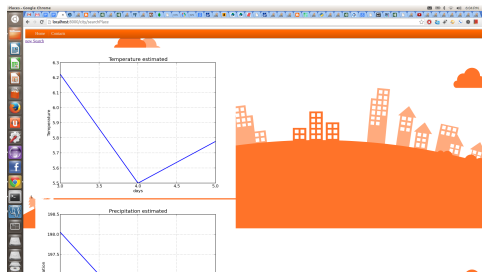


Figure 14: Design model 1.

The Implementation section describes how system applications will be installed, deployed and transitioned into an operational system for the purpose of our project. In this section we will give an overview of the system, describe major tasks involved in the implementation process, and the overall resources needed to support the implementation effort such as hardware, software, facilities, techniques, etc. Furthermore, we will present users with the documentation and tools necessary to make an informed decision on how to deploy our system for short-term vacationing. In addition, multiple features have been added and we will explain how

to utilize them in our system during its implementation process. We will demonstrate how these applications, based on our interface and database sub-section, can be installed and configured in any environment by deploying the enabled operations of our system that have been completely developed and tested for our project.

## 4.1 Interface

### 1. Features

- **Registration**

In order for users to perform any kind of task they need to sign into our system through their Facebook. Once they are into our system, we get access to their basic information such as name, gender, age, and others. Django easily connects python to facebook in order to process the necessary information for a given query.

- **Search**

In this feature we are searching into our database through Django for a particular city. This is done by entering the city name in Django, which queries into the database class.

- **Graph**

The graph is generated through the function that we have built into Django. This takes the regression of the data of each day within a given range. Then, we plot the graph with the resulting data using python. This data is an approximation of the weather that the user has selected for vacationing purpose.

- **Suggested List**

Every time the user searches for a city, this search is being saved by Django in a separate table that belongs to the user. Also, it stores the average temperature of the queried destination. Next, we query into our database using the average temperature to suggest for an appropriate destination for the user.

These components were planned, analyzed, and designed into our system through documentation such as data flow diagrams and others that will guide users to use our system if any issue arises. We have tested our system several times to make sure that our system works and performs the duties it is intended to perform. The main goal of our testing process was to detect any problem or error as early as possible in the implementation process. Therefore, it was critical to make sure that each application was implemented in accordance with our design and architecture plan to ensure that our system is ready for implementation. In this way, we were able to determine issues, occurred during the implementation process, which affected our system. In summary, we have established an overall development with the best practices to achieve the goal of our system implementation.

## 5. LESSONS LEARNED

## 5.1 Techniques

We learned numerous database and data mining techniques while working on this project. We learned a number of ways of formatting our data so that we can easily use it in our application. The data downloaded from the website was raw and faulty we added dates corresponding to each row to make it more meaningful. We related each longitude and latitude pair to a city. We learned how to convert text files into mysql database so as to be able to use in our project.

We learned how to query on a huge database. As a result of querying for the weather conditions of a place we get a number of results we use regression techniques on the resultant data to provide for a single valued for each corresponding date for vacationing.

We learned a few data mining techniques too. We were able to suggest a few vacation destination to our users to make our application more interactive. The suggestions were based on the age, gender and the last search using these variable we search in our database for more destinations.

## 6. CURRENT STATUS

The current status of our project is being explained through the three main sections that highly influenced the development of our project. First, we gathered the weather data of a few cities of U.S.A. This data consists of temperature, precipitation and wind speed corresponding to each city from 1980 to 2013. Each city is related to an id and a latitude and longitude.

Second, we developed an interface to provide the user with a GUI to query efficiently on the database. The interface asks the users to sign-in to proceed searching for destinations to vacation in a given range of dates. The sign-in is done through Facebook, which allows the user to avoid the whole signing in up process. Once signed-in, the user can choose a place to vacation and the vacation time duration. Then, the application searches for the corresponding weather data of the place and displays the approximate temperature, precipitation and wind speed of the place through graphs.

Lastly, we use some data mining techniques in our project too once the search result is displayed. In particular, we display some suggested places where the user may want to vacation other than the one the user has searched for. In fact, list of suggested cities provides for an interactive application.

## 7. FUTURE WORK

Big Data:

More research is needed to improve the ability to blend different datasets, and or data sources into integrated systems. As new types of data could be assimilated into different models, it will also be important to understand the error characteristics of our current data and our model to come up with modifications that may improve our system. Data assimilation for climate purposes is still in an early stage of development and requires continued research support. As these developments occur, reprocessing of data to take ad-

vantage of the new knowledge will be vital if we want to sustain long-term records in the future.

Data mining:

If our weather observations were well-documented, and it contains good metadata about the systems and networks used to make them, it would become more valuable with time. The creation of weather-quality data records is a fundamental objective for our observing system for weather. International standards and procedures for the storage and exchange of metadata need to be developed and implemented for many weather observing system components, including those of the operational satellite community. For this, it will be essential that all such data be properly archived and managed with the full expectation that they will be reused many times over in the future, often as a part of reprocessing or reanalysis activities. Future modifications may require that data be migrated to new media as technology changes, be accessible to users, and be made available with minimal incremental costs. For this, we will need systems for quality control and feedback, proper data archival and access, sustained data preservation, rescue and digitization, and climate data management systems. Capacity-building is needed for all of these.

## 8. CONCLUSION

This project update of the Best Time To Vacation was prepared in response to avoid unexpected bad weather during a vacation trip. The full implementation of time project, over the coming years, will ensure that users to have the observational information needed to understand, predict, and manage their response to weather and weather change, over the 21st century and beyond, for vacationing purposes. It will address the commitments of the users to certain places and support their needs for weather observations in fulfillment of the objectives of their vacation.

The database sustains observations of the essential climate variables that are used to make significant progress in the generation of weather analysis and derived information for our project. These observations give an enhanced support to our research, modeling, and analysis for users' s vacationing events. In fact, these observational records improve weather predictions for short-term vacations. These observations must be sustained into the future to evaluate how the weather is changing so that informed decisions can be made by any user on prevention, mitigation, and strategies when vacationing. It is crucial for users to support the further research needed to refine understanding of the weather approximated and its changes. This will allows us to initialize predictions on time scales out to decades ahead, and to develop the models used to make these predictions and short-term scenario-based projections.

## 9. REFERENCES

- [1] NCDC. Babel, Whether Datasets. *cdo-web*, 12(2):291–301, June 1991.  
<http://www.ncdc.noaa.gov/cdo-web/>
- [2] M. Clark. Post congress tristesse. In *TeX90 Conference Proceedings*, pages 84–89. TeX Users Group, March 1991.



- [3] M. Herlihy. A methodology for implementing highly concurrent data objects. *ACM Trans. Program. Lang. Syst.*, 15(5):745–770, November 1993.